

# **Encryption and Error Correction in Digital Communications**

**EE 463**

**Joe Antrim**

**4/1/2026**

|                                    |    |
|------------------------------------|----|
| 1. Introduction .....              | 2  |
| 2. Error correction .....          | 3  |
| Bit Parity.....                    | 3  |
| Triple Modular Redundancy .....    | 5  |
| Hamming Codes.....                 | 6  |
| Reed Solomon Codes .....           | 11 |
| Experimentation:.....              | 14 |
| 3. Encryption .....                | 17 |
| Caesar Cipher.....                 | 18 |
| Data Encryption Standard .....     | 18 |
| Advanced Encryption Standard ..... | 19 |
| Rivest-Shamir-Adleman .....        | 20 |
| Experimentation.....               | 20 |

# 1. Introduction

The age of modern computers has created a dependency on digital communication. Messages are transmitted through the open-air channel where they face interference and noise disrupting the continuity of the signal. While in the open-air channel, the signal may be received by an unintended and unprivileged agent. To validate the message received by the intended agent, the message is padded with additional information in the form of Encryption and error correction codes.

## 2. Error correction

Error correction involves the addition of bits to allow for an error in a message segment to be found and corrected without the need to retransmit the data.

### Bit Parity

Bit parity is a trivial method of detecting if a single error has occurred in the transmitted segment. If an even number of errors occur, bit parity will not be able to detect the presence of an error. Bit parity will also not be able to distinguish a single error from any odd number of errors.

Bit parity can come in either even or odd, where each respectively appends a single bit to ensure that the number of '1's in the segment is either even or odd. Like all error correction methods, the variation of either even or odd parity must be established before communication begins.

*Message: 1000 101*

Given the 7-bit message above, a parity can now be added.

*Even Parity: 1000 1011*

*Odd Parity: 1000 1010*

If during transmission the Most Significant Bit (MSB) is flipped the following signals will be received:

*Even Parity: 0000 1011*

*Odd Parity: 0000 1010*

In both cases, once received the messages will fail their respective parity. Though the location of error is unknown, the known presence of an error will allow that segment to be retransmitted.

If a second error was to occur, in for example, the 3<sup>rd</sup> MSB the following messages would be received.

*Even Parity: 0010 1011*

*Odd Parity: 0010 1010*

Each of these segments would pass their corresponding parity, and the parity bit once removed would have provided the incorrect message.

*Recieved Message: 0010 101*

Given a Bit Error Rate of  $1 > E > 0$ , the chance for a single error in the 8 transmitted bits is,

$$\frac{8!}{1!(8-1)!} E^1 (1 - E)^{8-1}$$

Which simplifies to,

$$8E(1 - E)^7$$

For very small values of  $E$  is approximately  $8E$ .

For exactly 2 errors it is given by,

$$\frac{8!}{2!(8-2)!} E^2 (1 - E)^{8-2}$$

Simplifying to

$$28E^2(1 - E)^6$$

Which is approximately  $28E^2$ .

Even for an extremely high bit error rate of 0.001, the chance of two errors is ~300 times less likely than a single error. Given a more realistic BER of  $1 \cdot 10^{-9}$  the chance of two errors is ~300 million times less likely to occur than a single error.

The efficacy of parity bits is dependent on the segment size. Given the same error rate, but instead using a 15-bit data segment gives a relative reduction of 2 errors of only a factor 133 million while

increasing the total number of these undetectable errors by a factor of 4.29.

Overall, bit parity increases the transmission size by 14.3% and can only identify errors that then must be retransmitted.

## Triple Modular Redundancy

By sending 3 copies of the message, any error occurring in only 1 one of the copies can be both identified and corrected. This method requires comparing each bit of all three copies and choosing the dominate version. This requires no padding of additional information, only the triplicate transmission.

Given an example message:

*Message:* 1010 0110

Three duplicates will be sent:

*Copy1:* 1010 0110

*Copy2:* 1010 0110

*Copy3:* 1010 0110

In this example, copy #2 will be completely corrupted.

*Copy2:* 0000 0000

By comparing each bit of all three copies and taking what is agreed upon by at least two of the copies, we can fully reconstruct the original message.

The Triple modular redundancy method is independent of message length and has no need for segmentation. For any single bit in a message, an error is allowed in 1 of the copies. Multiple errors can occur in each copy, but as each bit position only suffers a single, the message can be fully reconstructed.

A message can be reconstructed if one or zero errors occur in the 3 bits across the copies. If 2 or 3 errors occur the message will not be successfully reconstructed. Given a Bit Error Rate of  $1 > E > 0$ , the chances of having 2 and 3 errors is as follows:

2 Errors:

$$\frac{3!}{2!(3-2)!}E^2(1-E)^{3-2}$$

Simplifies to

$$3(E^2 - E^3)$$

3 Errors:

$$\frac{3!}{3!(3-3)!}E^3(1-E)^{3-3}$$

Simplifies to

$$E^3$$

The combined sum of unacceptable errors is

$$3E^2 - 2E^3$$

The extremely high BER of 0.001 the BER of the message is  $3 \cdot 10^{-6}$ . For a bit error rate of  $1 \cdot 10^{-9}$  the corrected error rate becomes  $3 \cdot 10^{-18}$ .

Triple modular redundancy is very effective at eliminating errors in a message and does not require retransmission of data. Triple modular redundancy increases message size by 200%, cutting bandwidth by a factor of 3.

## Hamming Codes

Hamming codes rely on the idea of multiple parity bits to act as a binary search through the data block to find and resolve a single error or identify the presence of 2 errors.

The transmitted data must be in the form of a matrix with dimensions following powers of 2. The matrix does not need to be square, but with a square matrix the ratio of data bits to parity bits is at its highest.

Parity bits are on the first row and column and appear at each first occurrence of a '1' in those indexes binary equivalent. An additional parity bit is found at the start of the matrix.

Coverage of the P1 parity bit:

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | P0 | P1 | P2 | D  |
| 01    | P3 | D  | D  | D  |
| 10    | P4 | D  | D  | D  |
| 11    | D  | D  | D  | D  |

Coverage of the P2 parity bit:

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | P0 | P1 | P2 | D  |
| 01    | P3 | D  | D  | D  |
| 10    | P4 | D  | D  | D  |
| 11    | D  | D  | D  | D  |

Coverage of the P3 parity bit:

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | P0 | P1 | P2 | D  |
| 01    | P3 | D  | D  | D  |
| 10    | P4 | D  | D  | D  |
| 11    | D  | D  | D  | D  |

Coverage of the P4 parity bit:

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | P0 | P1 | P2 | D  |
| 01    | P3 | D  | D  | D  |
| 10    | P4 | D  | D  | D  |
| 11    | D  | D  | D  | D  |

Coverage of the P0 parity bit:

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | P0 | P1 | P2 | D  |
| 01    | P3 | D  | D  | D  |
| 10    | P4 | D  | D  | D  |
| 11    | D  | D  | D  | D  |

This 4x4 set up is referred to as the extended Hamming(15,11) code as there are 15 bits (other than P0), 11 of which are designated for data.

To demonstrate the Hamming code, we will use the 11 data bits to store the current year, 2026.

$$\text{Message: } 111\,1110\,1010_2 = 2026_{10}$$

Filling that into our matrix from left to right.

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | P0 | P1 | P2 | 1  |
| 01    | P3 | 1  | 1  | 1  |
| 10    | P4 | 1  | 1  | 0  |
| 11    | 1  | 0  | 1  | 0  |

We can now set the P1 parity bit to obtain even parity in its coverage.

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | P0 | 0  | P2 | 1  |
| 01    | P3 | 1  | 1  | 1  |
| 10    | P4 | 1  | 1  | 0  |
| 11    | 1  | 0  | 1  | 0  |

After repeating for P2-4.

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | P0 | 0  | 1  | 1  |
| 01    | 1  | 1  | 1  | 1  |
| 10    | 0  | 1  | 1  | 0  |
| 11    | 1  | 0  | 1  | 0  |

With the P1-4 parity bits set, we can find and correct any singular error without the need for that P0 parity bit. By adding the P0 parity bit, we can both make the total transmission into a power of 2 (16-bit transmission segment in this case) and allow for the detection of a second error.



| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | 0  | 0  | 1  | 1  |
| 01    | 1  | 1  | 1  | 1  |
| 10    | 0  | 0  | 1  | 0  |
| 11    | 1  | 0  | 1  | 0  |

If this is the received data, we can check the parity of P1-4 and see that P1 and P4 fail.

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | 0  | 0  | 1  | 1  |
| 01    | 1  | 1  | 1  | 1  |
| 10    | 0  | 0  | 1  | 0  |
| 11    | 1  | 0  | 1  | 0  |

This limits the error to four positions. Looking at the successful P2 and P3 parity checks we can further deduce the location.

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | 0  | 0  | 1  | 1  |
| 01    | 1  | 1  | 1  | 1  |
| 10    | 0  | 0  | 1  | 0  |
| 11    | 1  | 0  | 1  | 0  |

Taking the inverse of that selection gives a different set of possible errors.

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | 0  | 0  | 1  | 1  |
| 01    | 1  | 1  | 1  | 1  |
| 10    | 0  | 0  | 1  | 0  |
| 11    | 1  | 0  | 1  | 0  |

Finding the intersection of the two sets pinpoints the exact error. Given the nature of binary, all that needs to be done to repair the error is to flip the bit at the given position.

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | 0  | 0  | 1  | 1  |
| 01    | 1  | 1  | 1  | 1  |
| 10    | 0  | 0  | 1  | 0  |
| 11    | 1  | 0  | 1  | 0  |

In the case of two errors.

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | 0  | 0  | 1  | 1  |
| 01    | 1  | 1  | 1  | 1  |
| 10    | 0  | 0  | 1  | 0  |
| 11    | 1  | 1  | 1  | 0  |

P1, P2, and P4 are successful, and only P3 fails. Where success are marked in green and its implied failure in orange on the first matrix and failed cases in next.

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | 0  | 0  | 1  | 1  |
| 01    | 1  | 1  | 1  | 1  |
| 10    | 0  | 0  | 1  | 0  |
| 11    | 1  | 1  | 1  | 0  |

| Index | 00 | 01 | 10 | 11 |
|-------|----|----|----|----|
| 00    | 0  | 0  | 1  | 1  |
| 01    | 1  | 1  | 1  | 1  |
| 10    | 0  | 0  | 1  | 0  |
| 11    | 1  | 1  | 1  | 0  |

This isolates the error to occur in the position (00,01), however there was no error. By checking the P0 parity bit, we find that the total parity is even, suggesting that either 0 or 2 errors occurred. Since the other parity bits claim to have found an error, we can deduce that 2 errors have happened, and we must retransmit the data block.

Hamming codes are typically used in 256 blocks where only 9 bits are needed for parity in the extended code.

Given an error rate  $0 < E < 1$  the chance for a single and double error are given respectively as:

$$\frac{256!}{1!(255!)} E \cdot (1 - E)^{255}$$
$$\frac{256!}{2!(254!)} E^2 \cdot (1 - E)^{254}$$

Which for small values of  $E$ , will approximate the form:

$$256E$$
$$\frac{67575}{2} E^2$$

The Hamming code process can be coded with this use of parity bit logic. The Hamming code process was coded in Python for this project and can be found in the GitHub repository with the name `hammingCode.py`.

<https://github.com/mrfroggyjoe/Digital-Communications>

## Reed Solomon Codes

Like all other error correction codes, additional data is added such that a clear pattern is formed. This pattern must be clear enough that it can be determined when parts of the data are altered. Reed Solomon codes are based on Lagrange polynomials, where in general  $n$  points can be intersected by a polynomial of degree  $n - 1$ . Lagrange polynomials are unique; there is only one polynomial of degree  $n - 1$  that dissects  $n$  points.

Reed Solomon codes use chunks of binary to represent positive integers as a symbol. These symbols are used in the construction of a Lagrange polynomial.  $k$  symbols will be used to construct the Lagrange polynomial of order  $k - 1$ . There will be  $t$  added symbols for redundancy such that each transmission block is  $n = t + k$ . The  $t \geq 2$  symbols are added as existing points in the  $k - 1$  polynomial.

When the  $n$  symbols are received, the first  $k$  points are used to create a polynomial. If the remaining  $t$  points fit onto the generated

polynomial, then no errors are present. If the remaining points do not fit on the polynomial, an error has occurred. If  $t-1$  points or less do not fit then the error occurs outside of the message and no action is required. To correct an error inside of the message, other combinations of  $k$  numbers need to be used to make a new polynomial until  $t-1$  or less discrepancies are found. Once the correct polynomial has been found, it can be used to replace the corrupted symbols. Since the binary is used to represent a symbol, Reed Solomon codes are able to correct entire symbols, allowing for multi-bit successive errors to be resolved.

An example of a Reed Solomon code is necessary given the conceptual complexity.

Our example message will be the first 4 prime numbers,  $\{1,2,3,5\}$ , which in a 3-bit binary can be represented by the bit string:

$$\text{Message} = 001\,010\,011\,101$$

For conceptual convience, we will choose each symbol to be 3-bits, giving us  $k = 4$  symbols. To have a proper code, we need at least  $t = 2$  additional symbols for a total of transmission  $n = 6$  symbols and 18 bits. With  $k = 4$  we will create a 3rd order Lagrange polynomial to choose the values of the additional symbols.

The easiest way to construct a Lagrange polynomial is to start with precalculated polynomials that are equal to 1 at an index 0 to  $k-1$  and zero for all other  $k$  indices. Each precalculated polynomial can be multiplied and summed by each symbol:

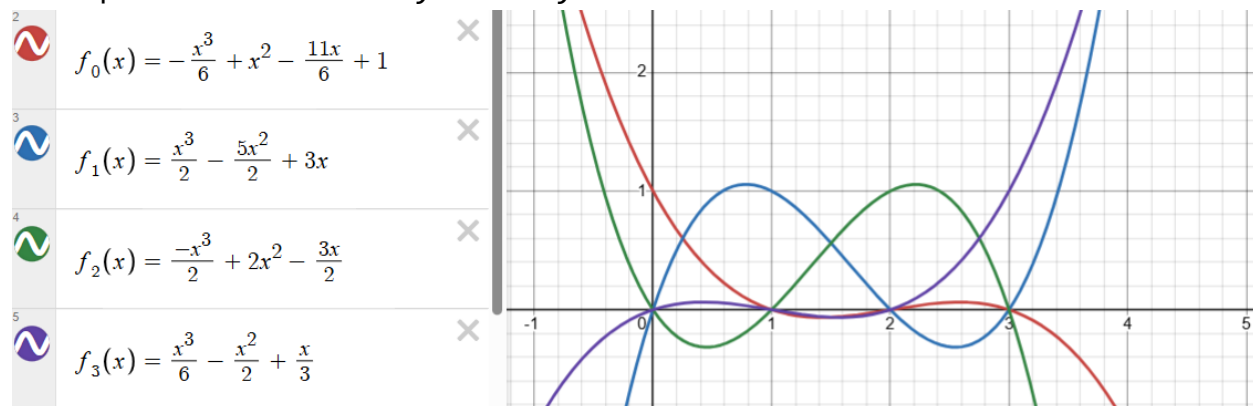


Figure RS.1 the 4 unique polynomials for a 4 symbol Reed Solomon code.

$$\text{Lagrange Polynomial} = \sum_{i=0}^{k-1} S_i \cdot P_i(x)$$

Using the summation formula for the first 4 prime numbers, we get a Lagrange polynomial for the data and may now get the 2 additional symbols.

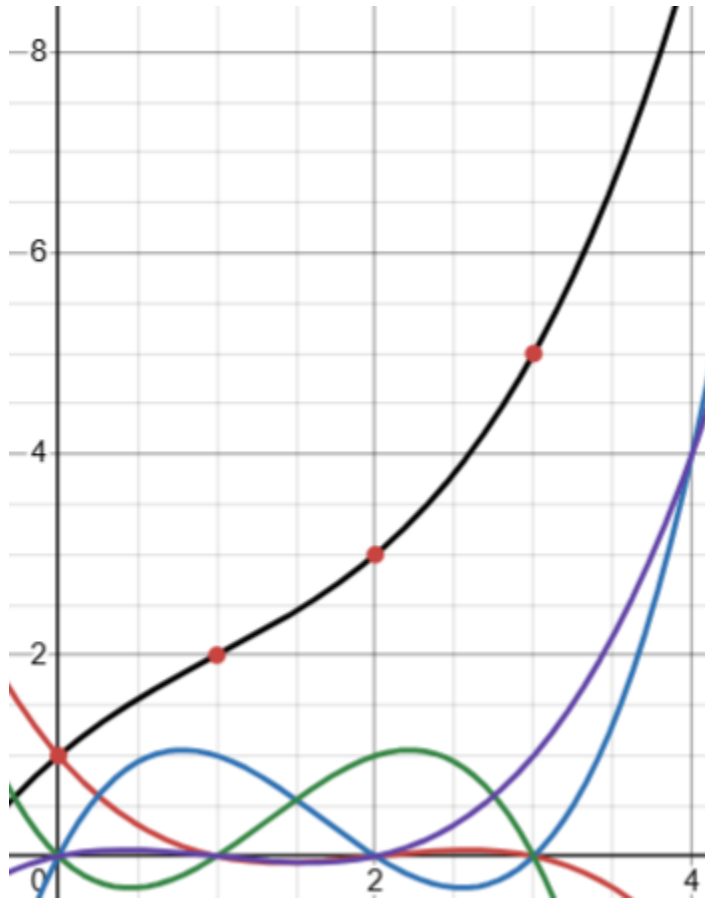


Figure RS.2 graph of finalized polynomial.

By evaluating the function at the additional  $t$  points, we get  $t_1 = (4,9)$ :  $t_2 = (5,16)$ . Since the values of the additional points are greater than the max value of the 3-bit symbol, we will modulo the value by 8 to ensure that it fits, giving 1 and 0 for each point respectively. These 6 symbols become the following transmission sequence.

001 010 011 101 001 000

If a 3-bit symbol is corrupted in transmission such that the new received sequence is

001 111 011 101 001 000

Upon reception we can create a new Lagrange polynomial based on the first 4 symbols.

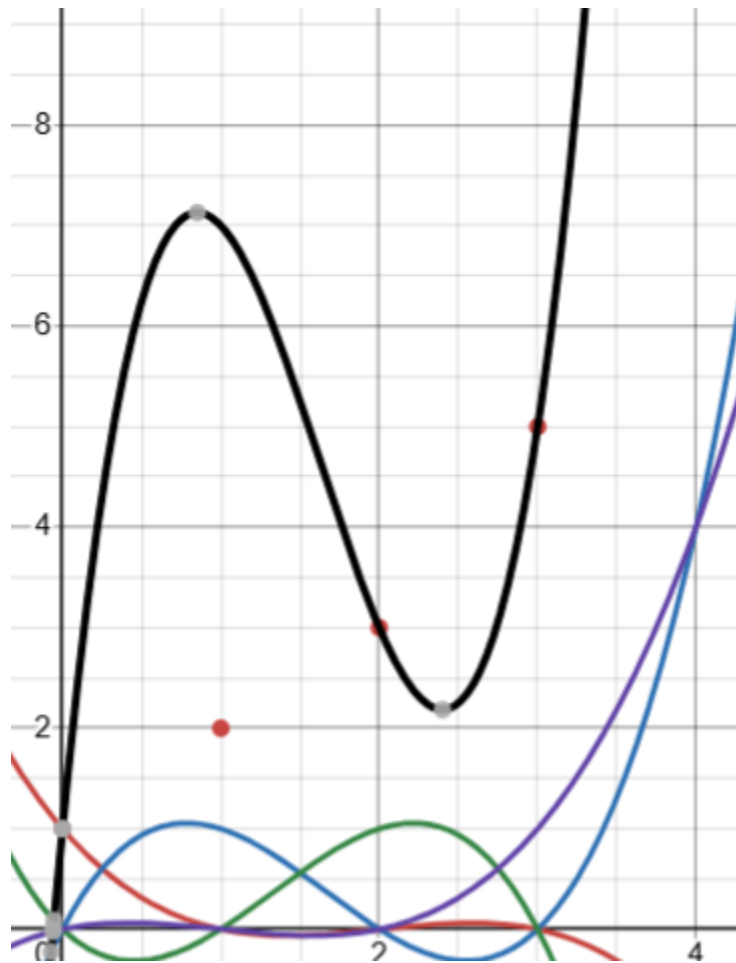


Figure RS.3 the newly constructed polynomial compared to the original points.

Based on the new polynomial, the next 2 values in the sequence should be 29 and 91 respectively, performing modulo we get 5 and 3, neither of which match the transmitted sequence. This means that an error has occurred in first  $k$  symbols. Creating a new Lagrange polynomial with the last 4 symbols and shifting it we get -55 and -14 which when modulo gives 1 and 2, 1 matches the transmitted sequence and 2 differs from the received 7. Since this is the only discrepancy, 7 must have been an error in transmission and the symbol can be replaced with 2.

## Experimentation:

This section will use the knowledge gained from the previous methods of error identification and correction to attempt to create new methods.

Error correction requires that some additional packets be added to create a pattern that describes the preexisting, and practically random sequence of data.

Bitwise operations such as AND and OR are often used as bit masks to isolate a few bits of a larger memory sequence. Since bitwise operations are per bit, there is no saving to increased segment size like in Hamming Codes or Reed Solomon Codes.

Given a 4-bit sequence:

0111

The minimized AND and OR bit masks would be:

OR: 1000

AND: 0111

Given the nature of the AND operation, it will always be the same as the message and is therefore no better than plain redundancy. The nature of the OR operation makes it the inverse of the message sequence and subsequently another form of redundancy.

The Exclusive Or (XOR) is another bitwise operation, with the unique property that the product of the operands, when XOR'ed with one operand, produces the other operand.

$Op_0 = 1010$

$Op_1 = 0011$

$Op_0 \oplus Op_1 = P_1 = 1001$

If the transmitted data is assembled as 2 data packets in the form of operands with an additional packet of the product, any sized error located to a single packet can be found. Using the example operation above with an error in the most significant bit of the seconds operand would produce the following 3 products:

Were the first operand after transmission is now:

$Op_0 = 1011$

$$Op_0 \oplus Op_1 = 0001$$

$$Op_0 \oplus P_1 = 0010$$

$$Op_1 \oplus P_1 = 0011$$

Since none of the resulting operands are equal to the transmitted values, it is clear that an error has occurred in transmission.

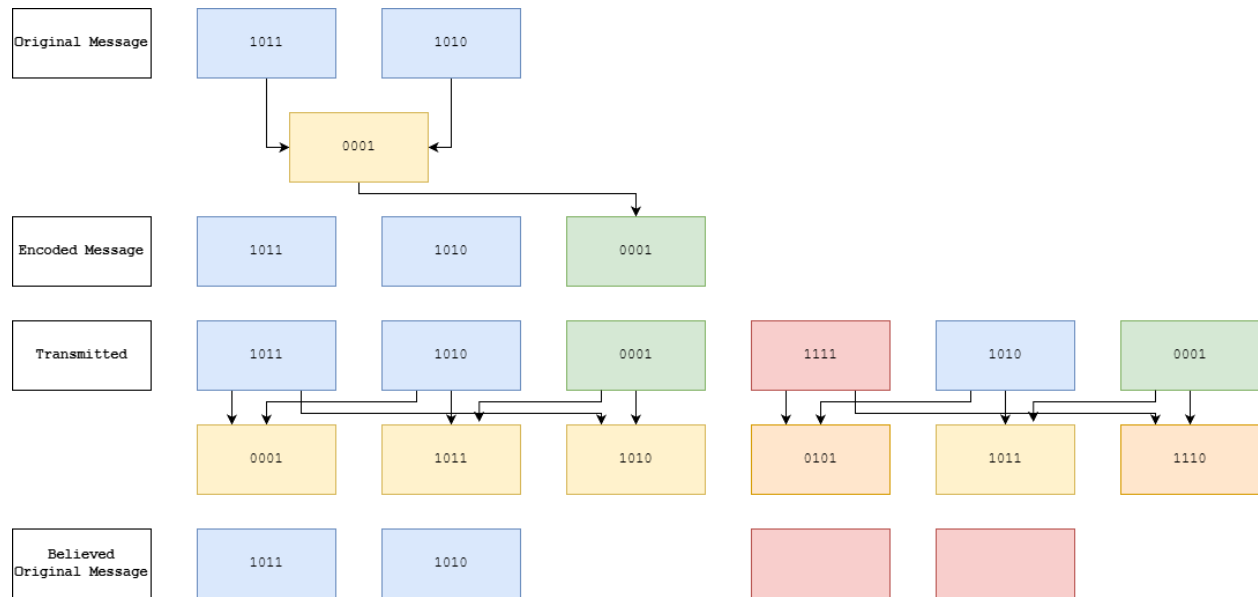


Figure EXED.1 Path of XOR operations to detect an error in a 3-symbol system, where the final 3 products are a rearrangement of the 3 transmitted symbols, and if not, an error has occurred.

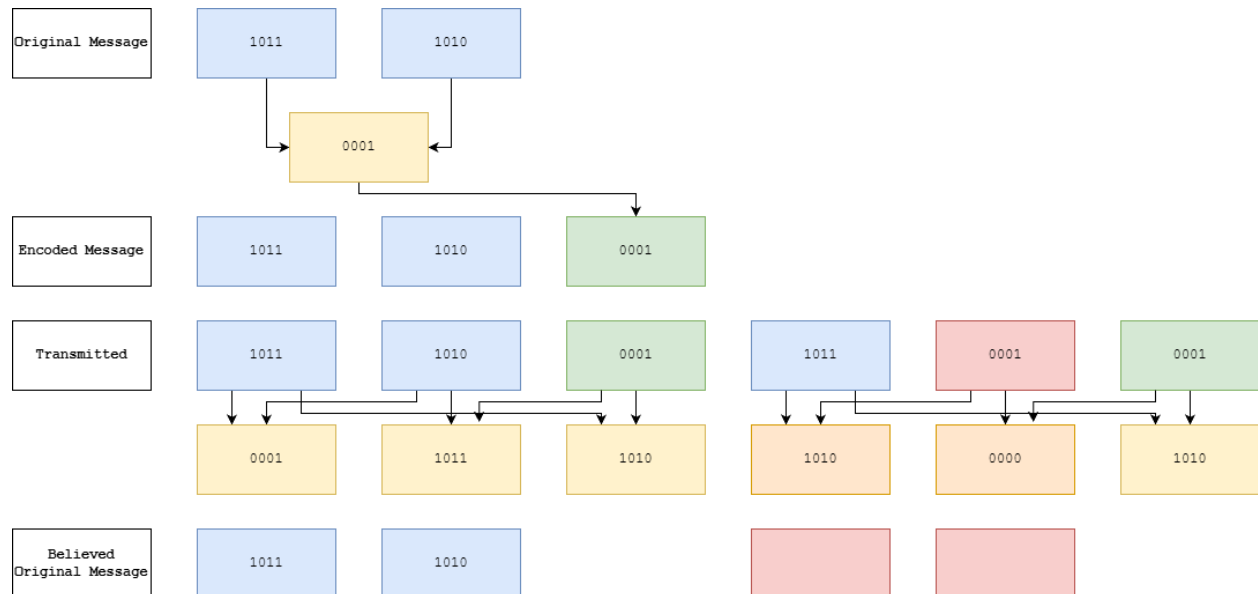


Figure EXED.2 Demonstration of full symbol error detection.



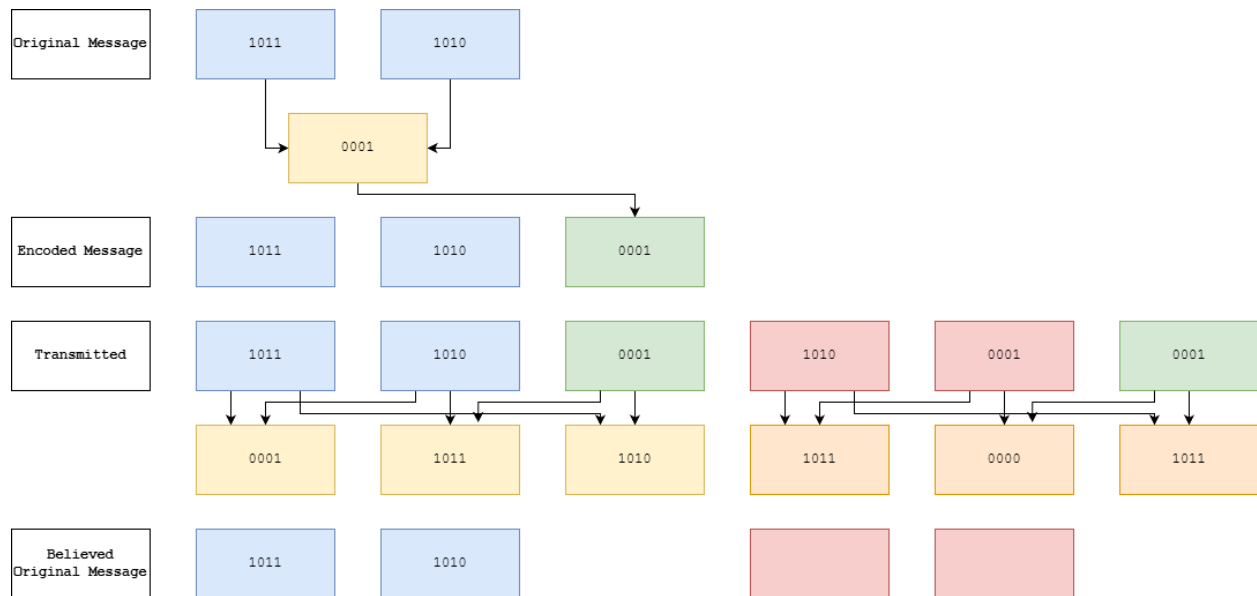


Figure EXED.3 Demonstration of two symbol error detection.

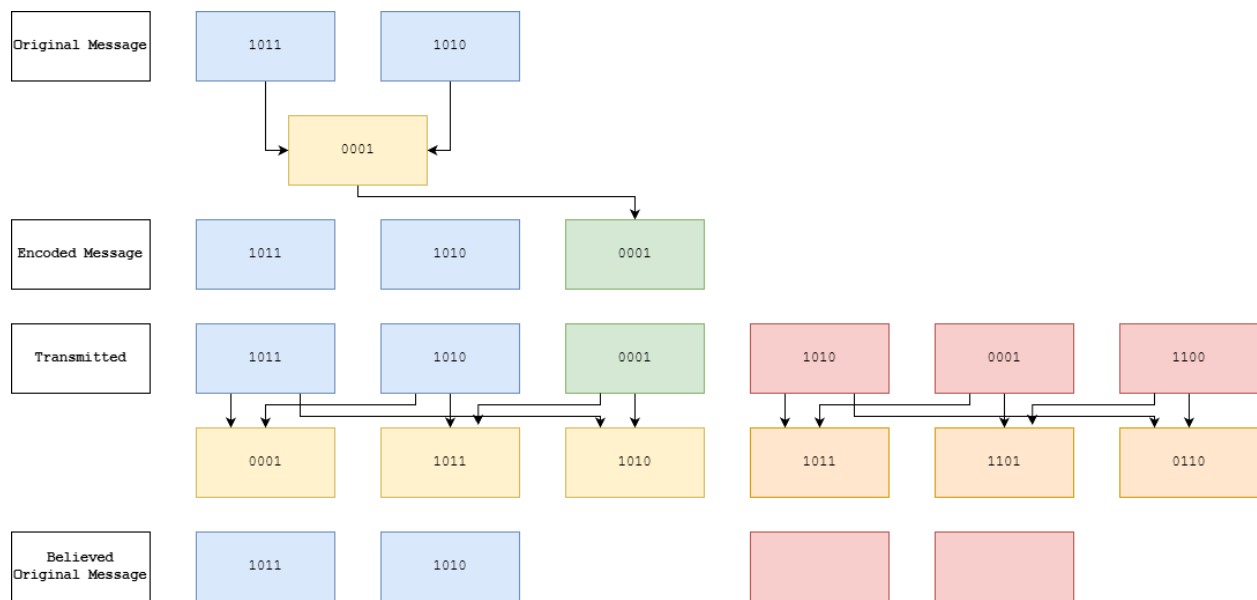


Figure EXED.4 Demonstration of errors in all symbols with successful error detection.

The redundant information is robust enough that any number of errors can be detected, though none can be corrected. If there are symbol errors that are equivalent to the swapping of any symbols, the error will not be detected. A one symbol error can always be detected; multi symbol errors can be detected in most cases.

### 3. Encryption

Wireless communications propagate regardless of the receiver and are therefore susceptible to interception. If critical information is sent, any number of bad actors may now know the privileged information. By encrypting the message using an agreed upon algorithm, the message can be safely sent and understood by only the intended recipient.

## Caesar Cipher

Caesar Ciphers are typically known to be used in written word communication, by shifting the letters by a specific number of times. A bit segment of a given size can be shifted either circularly or numerically.

For a circularly shifted Caesar Cipher the number of possibilities is equal to the number of bits in the segment - 1.

*Message:* 1011 0111

*Shift 1:* 1101 1011

*Shift 2:* 1110 1101

For a numerically shifted Caesar Cipher the number of options is  $2^n - 1$ .

*Shift + 1:* 1011 1000

*Shift + 2:* 1011 1001

For a numerically shifted Caesar Cipher to function the block size must be equal to or smaller than the interpreted data size. If a c style, 8-bit, char is used a 16 or 32-bit segment size would leave message data unshifted for relatively small cipher values.

## Data Encryption Standard

Data Encryption Standard is an older form of digital encryption that is symmetrical and relies on a private key for encryption and decryption. DES uses 64-bit data blocks and a 56-bit key. The method is reliant on a Feistel network, the exclusive or operation, and swapping of the left and right sides.

The swapping of the left and right sides allows the process to be symmetrical, allowing the same hardware and machine code to be used

for both the encryption and decryption of the message. Given the time of the algorithm's development, the reusability of code and machine space is critical for its standard adoption.

The 56-bit key, from a 64-bit key, is broken down into 16 48-bit subkeys that are used in each round. . In the case of decryption the list of subkeys is reversed, such that by repeating the same logic as encryption the message will be returned as the final result.

The core of DES is in the Feistel network, which takes a 48-bit subkey generated from a key schedule from the original 56-bit key along with a 32-bit message segment. The Feistel network adds nonlinearities with the use of the S-boxes and expansion systems, this makes the system much more difficult to break.

The Feistel network starts by expanding the 32-bit message into a 48-bit block, so it can be XORed with the sub key of the round. Each 6-bit segment is entered into the corresponding S-Box to undo the expansion returning to 8 4-bit segments that can be returned from the Network.

The return from the Feistel network is XORed with the other half of the message sequence. The left and right are swapped, and the process is repeated 16 times with each subkey. At the end of the 16 rounds there is an inverse permutation, and then the message may be sent.

A Python implementation of DES to encrypt and decrypt an image, and an audio file can be found in my GitHub repository under the name DES.py. <https://github.com/mrfroggyjoe/Digital-Communications>

## Advanced Encryption Standard

Advanced Encryption Standard is similar to DES; however, it relies on 128 data blocks and keys and a more intensive 'scrambling' of the data. The increase in key size to 128-bits greatly improves the security of the algorithm. The 128-bit key has 10 million trillion more possible combinations than a 64-bit key, making a brute force attack nearly impossible.

A Python implementation of AES to encrypt and decrypt an image, and an audio file can be found in my GitHub repository under the name AES.py. <https://github.com/mrfroggyjoe/Digital-Communications>

## Rivest-Shamir-Adleman

Rivest-Shamir-Adleman, known as RSA, is an asymmetric encryption and decryption method reliant on the modulo operator. RSA has a public encryption key and a private decryption key. Where the message  $m$  and the encrypted text  $c$  are related, as shown below.

$$c = \text{mod}(m^e, n)$$

$$m = \text{mod}(c^d, n)$$

The computational complexity of RSA encryption comes from the calculations of the keys.

First, 2 prime numbers,  $p$  and  $q$  are chosen.  $N$  is the product of  $p$  and  $q$ , and  $Z = (p-1)(q-1)$ . Another number  $e$  is chosen that it is coprime with  $Z$ . This is the public key used for encryption.

The private key  $d$  is chosen such that  $\text{mod}(e \cdot d - 1, n) = 0$

Once the keys have been calculated the messages can be encrypted and decrypted by performing exponential math and modulo operations as seen above.

## Experimentation

The key principle of encryption is to scramble the data non-destructively. For the system to be secure, some portions of the method must be kept secret such that the message is only comprehensible by the target receiver. Similarly to the experimentation in error detection, the non-destructive nature of the XOR operation is extremely useful.

By combining an XOR operation with a shift, equal to the number of '1's in the key, a message and key of any equal length can be encrypted. An 8-bit example is provided in figure EXC.1.

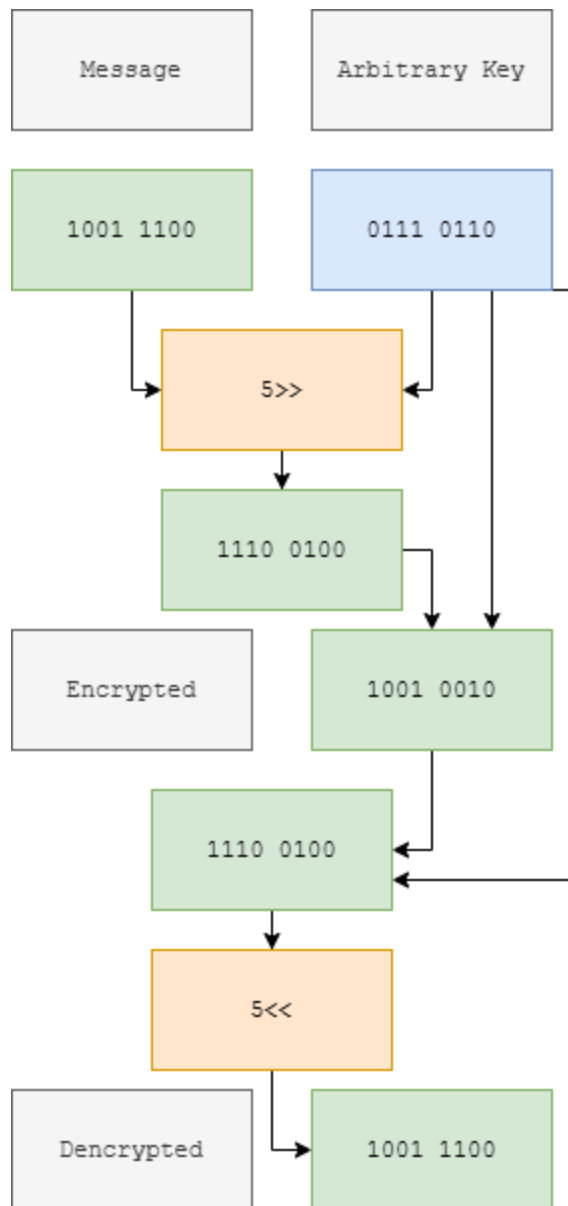


Figure EXC.1 a simple example of using bit shifts and XOR operations to encrypt a message.

This process, at this size is easily brute forced without knowledge of key. If the key and message size are increased and the process expanded with additional shifting and XOR operations, the method can become significantly more secure. A more secure process, based in this simple logic has been implemented in encrypt.py.

Without any sort of non-linear permutations, like the S-Boxes of DES, this method of encryption can still be easily broken.